

Parametric throughput analysis of synchronous data flow graphs

Citation for published version (APA):

Ghamarian, A. H., Geilen, M. C. W., Basten, T., & Stuijk, S. (2007). *Parametric throughput analysis of synchronous data flow graphs*. (ES reports; Vol. 2007-08). Technische Universiteit Eindhoven.

Document status and date:

Published: 01/01/2007

Document Version:

Publisher's PDF, also known as Version of Record (includes final page, issue and volume numbers)

Please check the document version of this publication:

- A submitted manuscript is the version of the article upon submission and before peer-review. There can be important differences between the submitted version and the official published version of record. People interested in the research are advised to contact the author for the final version of the publication, or visit the DOI to the publisher's website.
- The final author version and the galley proof are versions of the publication after peer review.
- The final published version features the final layout of the paper including the volume, issue and page numbers.

[Link to publication](#)

General rights

Copyright and moral rights for the publications made accessible in the public portal are retained by the authors and/or other copyright owners and it is a condition of accessing publications that users recognise and abide by the legal requirements associated with these rights.

- Users may download and print one copy of any publication from the public portal for the purpose of private study or research.
- You may not further distribute the material or use it for any profit-making activity or commercial gain
- You may freely distribute the URL identifying the publication in the public portal.

If the publication is distributed under the terms of Article 25fa of the Dutch Copyright Act, indicated by the "Taverne" license above, please follow below link for the End User Agreement:

www.tue.nl/taverne

Take down policy

If you believe that this document breaches copyright please contact us at:

openaccess@tue.nl

providing details and we will investigate your claim.

Parametric Throughput Analysis of Synchronous Data Flow Graphs

AmirHossein Ghamarian, Marc Geilen, Twan Basten, Sander Stuijk



ES Reports

ISSN 1574-9517

ESR-2007-08

1 November 2007

Eindhoven University of Technology
Department of Electrical Engineering
Electronic Systems



© 2008 Technische Universiteit Eindhoven, Electronic Systems.
All rights reserved.

<http://www.es.ele.tue.nl/esreports>
esreports@es.ele.tue.nl

Eindhoven University of Technology
Department of Electrical Engineering
Electronic Systems
PO Box 513
NL-5600 MB Eindhoven
The Netherlands

Parametric Throughput Analysis of Synchronous Data Flow Graphs ^{*†}

A.H. Ghamarian, M.C.W. Geilen, T. Basten, S. Stuijk
Eindhoven University of Technology, Electronic Systems Group
a.h.ghamarian@tue.nl

Abstract. *Synchronous Data Flow Graphs (SDFGs) have proved to be a very successful tool for modeling, analysis and synthesis of multimedia applications targeted at both single- and multiprocessor platforms. One of the most prominent performance constraints of concurrent real-time applications is throughput. For given actor execution times, throughput can be verified by analyzing the SDFG models of such applications, for instance using maximum cycle mean analysis or state space analysis. In various contexts, such as design space exploration or run-time reconfiguration, many fast throughput computations are required for varying actor execution times.*

We present methods to compute throughput of an SDFG where actor execution times can be parameters. The throughput of these graphs is obtained in the form of a function of these parameters. Recalculation of throughput is then merely an evaluation of this function for specific parameter values, which is much faster than the standard throughput analysis. We propose three different algorithms for parametric throughput analysis and evaluate these algorithms experimentally, showing the feasibility of the approach and showing that a divide and conquer algorithm performs best.

1 Introduction

Synchronous Data Flow Graphs (SDFGs, [10]) are a useful means for modeling and analysis of applications such as DSP applications and concurrent real-time multimedia systems [10, 13, 15, 18]. SDFGs have been used for both single and multiprocessor platforms. The main aim in such systems is realizing a predictable performance. SDFGs are equipped with several timing analysis techniques, which are used for evaluating performance metrics of such applications, most importantly throughput.

An SDFG is a graph where nodes are called *actors* and edges are called *channels*. Actors typically model application tasks. The worst case execution times of tasks are

assigned to actors as their execution times and edges model data communication and control dependencies.

Throughput analysis is a crucial indicator of performance used both at design time (e.g., in design space exploration, DSE) and run-time (e.g., resource management). In DSE many different settings of the system are explored [16, 17], which leads to many throughput calculations. At run-time, prediction of throughput is required for proper assignment of resources to applications during reconfigurations [14]. In both cases, throughput calculations need to be as fast as possible with very strict time and resource requirements for run-time applications. Another application example is the study of the impact of variation of execution times under production process variations on throughput [5].

Throughput analysis has been studied in the literature [2, 7] and different methods have been proposed. The execution times of actors are assumed to be fixed numbers in all of the proposed methods. Therefore, any change in the execution time of one or more actors of an SDFG leads to a recomputation of the throughput from scratch. However, calculating the throughput is expensive in many cases, and has exponential time complexity in the worst case.

In this report, we consider parametric SDFGs, a generalization of SDFGs where actors can have parameters as their execution times. We study three algorithms to calculate the throughput of a parametric SDFG as a simple function of the parameters. The resulting function gives the throughput of the SDFG for any value in the range of the parameters. The first two algorithms are variants of the standard throughput analysis algorithms for SDFGs for parametric actor execution times. The third algorithm is based on a divide and conquer (DC) strategy. In the experimental results, we compare the advantages and the drawbacks of these algorithms. The DC algorithm turns out to be the most efficient in practice.

Section 2 introduces synchronous data flow graphs, and their parametric extension. Sections 3, 4 and 5 explain the different methods for finding the throughput of parametric SDFGs. Comparison of the methods is done in Section 6. Section 7 concludes.

^{*}This work was supported by the Dutch Science Foundation NWO, project 612.064.206, PROMES.

[†]This report is an extended version of the paper A.H. Ghamarian, M. Geilen, T. Basten and S. Stuijk. Parametric Throughput Analysis of Synchronous Data Flow Graphs. In the proceedings of the conference on Design Automation and Test in Europe, DATE 2008, Proc. ACM, 2008". It provides the proofs omitted from the original paper.

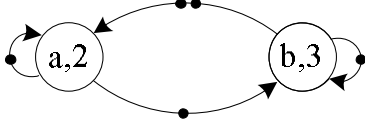


Figure 1. An example SDFG G_{ex} .

2 Synchronous Data Flow Graphs (SDFGs)

2.1 Basic Definitions

SDFGs are a natural means to capture concurrent DSP and multimedia applications. Tasks are modeled as *actors* and channels represent data dependencies. The execution of an actor is referred to as a *firing*, the data items communicated between actors are modeled by *tokens*, and the amounts of tokens produced and consumed in a firing are referred to as *rates*. Channel capacities are unbounded, i.e., channels can contain arbitrarily many tokens (but limited capacities can be modelled in the graph). To make SDFGs amenable to timing analysis, usually fixed execution times are associated to actor firings.

Figure 1, shows an example of a timed SDFG with two actors, a and b , with execution times 2 and 3 respectively, annotated inside the actors. The channel rates in this example are all one. Tokens in channels are shown by black dots.

We assume a set $Ports$ of ports, and with each port $p \in Ports$ we associate a positive finite rate $Rate(p) \in \mathbb{N} \setminus \{0\}$. An actor a is a tuple (In, Out, E) consisting of a set $In \subseteq Ports$ of input ports ($In(a)$), a set $Out \subseteq Ports$ of output ports ($Out(a)$) with $In \cap Out = \emptyset$ and $E \in \mathbb{R}$ representing the execution time of a ($E(a)$). An SDFG is a tuple (A, C) with a finite set A of actors and a finite set $C \subseteq Ports^2$ of channels. The source of every channel is an output port of some actor; the destination is an input port of some actor. All ports of all actors are connected to precisely one channel.

The execution of an actor is defined in terms of *firings*. When an actor a starts its firing, it removes $Rate(q)$ tokens from all $(p, q) \in C, q \in In(a)$. The firing then continues for $E(a)$ time and when it ends, it produces $Rate(p)$ tokens on every $(p, q) \in C, p \in Out(a)$. An SDFG with all port rates equal to one is called a *Homogenous Synchronous Data Flow Graph* (HSDFG). Any SDFG can be converted [15] to an equivalent HSDFG. However, this conversion may lead to an exponential explosion in the size of the SDFG [12].

Not all SDFGs with arbitrary rates are of practical interest, but only those which comply with an easy to verify condition called *consistency* [10]. Inconsistent graphs either deadlock or are unbounded [6].

Consistency is defined as follows.

Definition 1 [*Consistent SDFG, repetition vector*] A repetition vector q of an SDFG (A, C) is a function $A \rightarrow \mathbb{N}$ such that for each channel $(o, i) \in C$ from actor $a \in A$ to $b \in A$, $Rate(o) \cdot q(a) = Rate(i) \cdot q(b)$. A repetition vector a is called *non-trivial* if and only if $q(a) > 0$ for all $a \in A$.

An SDFG is called *consistent* if it has a non-trivial repetition vector. For a consistent graph, there is a unique smallest non-trivial repetition vector, which is designated as the *repetition vector* of the SDFG.

2.2 Throughput

An SDFG can have more than one execution. Firing every actor as soon as it gets enabled, *self-timed* execution, leads to the highest achievable throughput [15].

Definition 2 [*Actor Throughput*] The throughput $Th(a)$ of an actor a of an SDFG is defined as the average number of firings of a per time unit in the self-timed execution.

For brevity, we focus on strongly connected SDFGs. The extension of the results to general graphs can be done by combining the results of the strongly connected components of the SDFG [6].

In the literature, there are two different methods for calculating throughput of an SDFG.

HSDFG method: It is proven in [9] that the throughput of an SDFG is equal to the inverse of the maximum cycle mean (MCM) of the equivalent HSDFG [15]. The cycle mean λ of a cycle of an HSDFG is defined as the total execution time of the cycle over the number of tokens in that cycle. There are efficient algorithms for calculating the MCM of an HSDFG (see [2] for an experimental survey). However, the conversion of an SDFG to an equivalent HSDFG may lead to an exponential explosion in the size of the graph.

State-space method: Self-timed execution of an SDFG ends in a repetitive sequence of actor firings, the *periodic phase* of execution. The throughput of an actor can be calculated by dividing the length of the period by the number of firings of the actor in one period.

[7] compares both methods experimentally showing that the state-space method outperforms the HSDFG method.

2.3 Parametric SDFGs

Here we introduce a *parametric SDFG*, as an SDFG where the execution time of at least one of its actors is a parametric expression, where a parametric expression is a linear expression in terms of some parameters. In principle, parametric expressions can take any value of positive real

numbers. For example, G_{ex} of Figure 1 becomes a parametric SDFG, G_{ex}^{par} , if we assume that the execution times of a and b are given by parameters $p, q \in \mathbb{R}^+$. We are interested in the throughput of a parametric SDFG in the form of a function of the parameters: $f(p, q)$. The domain of this function, the set of values that the parameters can take, is called the *parameter space* of the graph, which is d -dimensional when the number of parameters is d . Evaluating this function for a point in the parameter space is computationally much cheaper than redoing any of the traditional throughput calculations for those parameter values. We formally define the parametric execution times in the following definition.

Definition 3 [Parametric Execution Time] Assume a d -dimensional vector of parameters. A parametric execution time models the execution duration of actors of a parametric SDFG, in terms of these parameters. In a parametric SDFG (A, C, E_p) , (A, C) is an SDFG and the execution time is a function $E_p : A \rightarrow T$ that associates with each actor $a \in A$ a linear combination of parameters, formally denoted by a vector t in $T = \mathbb{Q}^{d+1}$ containing the coefficients of the linear expression. For $a \in A$, $E_p(a)$ assigns the amount of time it takes to complete a firing of actor a .

We know that the throughput of an SDFG corresponds to the inverse of the maximum cycle mean of its equivalent HSDFG. The cycle mean of each cycle equals the sum of the execution times of actors in the cycle divided by the number of tokens on the cycle. Consequently, any cycle mean in a parametric HSDFG is a linear combination of parameters plus a constant, representing the non-parameterized actors in the cycle, which is 0 when there are no such actors. We call these linear combinations *cycle mean expressions*. $\frac{1}{3}p + \frac{1}{3}q + 0$ is a cycle mean expression of G_{ex}^{par} corresponding to the cycle through a and b . A cycle mean expression is represented by a vector $\bar{e}$ whose elements are the coefficients of the linear expression, e.g., $(1/3, 1/3, 0)$ in the example. If cycle c has cycle mean expression $\bar{e}_c$, then its cycle mean, λ_c , for each point $\bar{p} \in \mathbb{R}^d$ in the parameter space can be calculated by $\lambda_c(\bar{p}) = \bar{e}_c \cdot (\bar{p}, 1)$, where “ $\cdot$ ” is the inner product of two vectors. For example, for G_{ex}^{par} , $\lambda(1, 2) = (1/3, 1/3, 0) \cdot (1, 2, 1) = 1$. We denote the evaluation of a point $\bar{p}$ in a cycle mean expression $\bar{e}_c$, by $\bar{e}_c(\bar{p})$. The maximum cycle mean of an HSDFG G for each point $\bar{p}$ in the parameter space, denoted by $\lambda^*(\bar{p})$, can be calculated via

$$\lambda^*(\bar{p}) = \max_{c \in \text{cycles}(G)} \bar{e}_c(\bar{p}).$$

Note that λ^* is a continuous function as it is the composition of continuous functions $\max$ and $\bar{e}_c$. Any expression that has the maximum cycle mean value for some point is called a maximum cycle mean expression (*mcme*). Any

mcme that has the maximum cycle mean value for some point for which no other mcme has the maximum value is a *dominating mcme*. It is shown below that the dominating mcmes are sufficient to compute λ^* . The other cycle mean expressions, including the other mcmes, are called *redundant expressions*.

Definition 4 DCMS Given an HSDFG, the dominating cycle mean set (DCMS) is the set of dominating mcmes.

Proposition 5 Given an SDFG G and a parameter space, the maximum cycle mean of G for any value of parameters can be obtained by its DCMS.

$$\lambda^*(\bar{p}) = \max_{\bar{e} \in \text{DCMS}} \bar{e}(\bar{p}).$$

PROOF Suppose the parameter space has d dimensions. For any arbitrary point $\bar{p}$ in the parameter space, if only one mcme is maximum at $\bar{p}$ then that mcme is in DCMS by definition. If only two mcmes $\bar{e}_1$ and $\bar{e}_2$ are maximum at $\bar{p}$, then $\bar{p}$ can only lie in the plane characterized by $\bar{e}_1(\bar{p}) = \bar{e}_2(\bar{p})$ which has less than d dimensions as the mcmes are linear combinations of parameters. In other words, $\bar{p}$ is on the border of the regions in parameter space for which $\bar{e}_1$ or $\bar{e}_2$ are mcmes. So, there is a region around $\bar{p}$ in the parameter space for which either $\bar{e}_1$ or $\bar{e}_2$ is uniquely maximum. Therefore, at least one of $\bar{e}_1$ or $\bar{e}_2$ belongs to the DCMS. If more than two mcmes are valid at $\bar{p}$, we can use a similar argument to show that at least one of them belongs to the DCMS. We showed that for any point $\bar{p}$ in the parameter space at least one dominating mcme belongs to the DCMS. $\square$

Note that when we talk about the DCMS of an SDFG we refer to the DCMS of its equivalent HSDFG. Conversion of a parametric SDFG to an equivalent HSDFG can be done using the algorithm for non-parametric graphs, since execution times have no impact on the conversion algorithm. Thus, throughput analysis for a parametric SDFG can be done by finding its DCMS. This minimum set can be obtained from the set of all expressions/mcmes by removing all redundant expressions. Checking the redundancy of an expression is equivalent with checking the infeasibility of a system of linear equations [4]. However, since in our case the values of parameters are positive, there is a fast way to remove a large part of the redundant expressions. An expression is redundant if all of its coefficients are less than or equal to those of another expression. In other words, if we look at the vectors $\bar{e}$ of the expressions, then all points (expressions) are dominated by a subset of points with larger coefficients. Those points which are not dominated by other points using this redundancy check are called *pareto points*. In other words, *pareto points* are dominating all non-*pareto points*. We denote $\bar{e}_1 \preceq \bar{e}_2$ to express that $\bar{e}_1$ is dominated by $\bar{e}_2$. A *pareto dominance* test is much easier than the general redundancy checks.

Although finding the *pareto set* of expressions often removes a large part of the redundant expressions, the *pareto*

set is not necessarily the DCMS. For example, suppose our set of expressions is $\{p, q, (p + q)/3\}$, or, in terms of vectors $\{(1, 0, 0), (0, 1, 0), (1/3, 1/3, 0)\}$. Even though $(1/3, 1/3, 0)$ is a pareto point (it is not dominated by either $(1, 0, 0)$ or $(0, 1, 0)$), there is no point in the parameter space where $(p + q)/3$ has a larger value than all other expressions, making it redundant. Nevertheless, pruning the set of expressions via a pareto dominance test before applying any general redundancy test is worthwhile. Figures 2(a) and 2(b) visualize this set of throughput expressions and their related mcme for parameter ranges of $[1, 5]$. The horizontal axes are parameters and the vertical axes give throughput and mcme values for Figures 2(a) and 2(b) respectively. As it can be seen from the figure, the red plane (expression $(p + q)/3$) is always dominated by either green (p) or blue (q) planes for any point in the parameter space (similar for the throughput expressions in Figure 2(a)).

Existing methods of calculating throughput for SDFGs are not directly applicable to parametric SDFGs. The efficient MCM analysis algorithms which work on HSDFGs cannot be applied on parametric SDFGs. The conversion of SDFGs to HSDFGs can easily be adapted though. Therefore, a naive MCM analysis leads to enumerating all simple cycles of the HSDFG and collecting the expressions in the DCMS. Also, the state-space method of [7] cannot be directly used for parametric throughput analysis, but it can be generalized. In the remainder, we introduce two variations of the existing methods and one new method for calculating the throughput of a parametric SDFG.

3 HSDFG Method

This section shows how a parametric throughput can be calculated using the conversion of an SDFG to an HSDFG. The MCM of an HSDFG can be found by enumerating all simple cycles. The cycle mean of each cycle can be calculated by summing up all of the execution times of actors in the cycle and dividing it by the number of tokens on the cycle. Finally, the DCMS of the parametric HSDFG is obtained by removing the redundant expressions as explained in Section 2.3. While enumerating the cycles and calculating their cycle mean expressions only the pareto points are kept.

The example in Figure 1 is already an HSDFG, so no conversion is needed. This graph has three simple cycles (a, a) , (b, b) and (a, b, a) with one, one and three tokens respectively. Therefore, the cycle mean expressions are p , q and $(p + q)/3$. Since all expressions are pareto points in the parameter space, no points get eliminated in the pareto test.

In the next step of the algorithm, we see that $(p + q)/3$ is redundant. This follows from the infeasible linear system $\{(p + q)/3 > p, (p + q)/3 > q\}$. Therefore, $DCMS(G) = \{p, q\}$ and $\lambda^*(p, q) = \max\{p, q\}$.

Algorithm HSDFG method(G)

Input: A parametric SDFG G

Output: DCMS of G

1. $DCMS = \emptyset$
2. Convert G to equivalent HSDFG H
3. **for** each simple cycle c in H
4. **do if** $\bar{e}_c \not\preceq \bar{e}_i$ for all $\bar{e}_i \in DCMS$
5. **then** remove all $\bar{e}_i$ from $DCMS$ for which $\bar{e}_i \preceq \bar{e}_c$
6. insert $\bar{e}_c$ in $DCMS$
7. Remove redundant expressions from $DCMS$
8. **return** $DCMS$

Note that finding a set of dominating expressions among expressions (Line 7 of the algorithm) has been solved efficiently in the context of determining the upper envelope of pairwise linear functions [3]. Since the time spent on this part of the algorithm is negligible compared to the first part, we used the straightforward redundancy check explained above in our experiments using a linear programming C library (LPsolve [1]).

4 State-Space Method

State-space-based throughput calculation for SDFGs [7] avoids the conversion to HSDFGs. This section generalizes the state-based method to calculate the throughput of a parametric SDFG.

4.1 State Space

The behavior of an SDFG can be defined in terms of a state transition system.

Definition 6 [State] *The state of a timed SDFG (A, C) is a tuple (γ, v) . γ associates with each channel the amount of tokens present in that channel in that state. To keep track of time progress actor status $v : A \rightarrow \mathbb{N}^R$ associates with each actor $a \in A$ a multiset of numbers representing the remaining times of different active firings of a .*

An actor consumes its required input tokens at the start of its firing, as soon as sufficient tokens are available. Output is produced at the end of the firing. Since channels have infinite capacity, sufficient space is always available. If we are interested in throughput, and not for example in functional analysis, we abstract from the actual communicated data.

In Figure 1, the initial state is $((1, 1, 1, 2), (\{\}, \{\}))$, where the first vector shows the token distribution of channels, starting from the self-loop channel of a and continuing counterclockwise. The second vector, v , is the vector of multisets of the remaining execution times of a and b . Initially, both multisets are empty. At this point, both

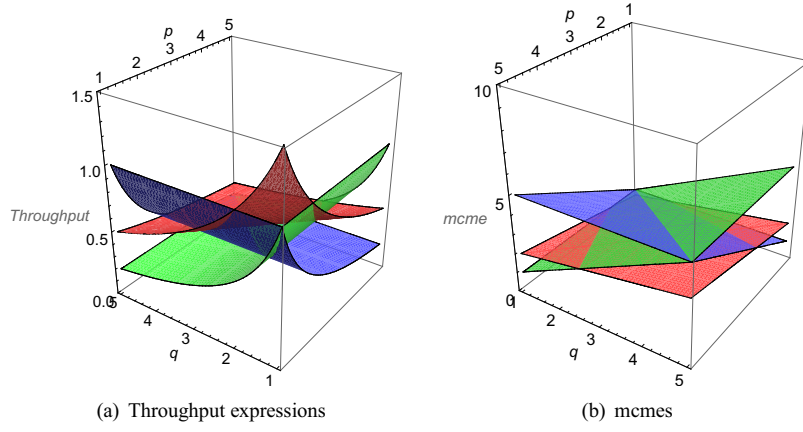


Figure 2. mcme and related throughput expressions of the running example.

a and b are enabled and they start their firings, changing the token distribution vector to $(0, 0, 0, 1)$ and the vector of remaining execution times v to $(\{2\}, \{3\})$. No more actor firings can occur before actor a finishes. So the time goes forward for 2 time units. Completing the firing of a leads to state $((1, 1, 0, 1), (\{\}, \{1\}))$. Firing actor a once again then results in $((0, 1, 0, 0), (\{2\}, \{1\}))$, after which time progresses for 1 time unit, b completes its firing, and so on.

We generalize this model to parametric SDFGs. In the state space of a parametric SDFG, v contains parametric elements, expressions in terms of the parameters. Since the relations between parameters are not known, we cannot always be sure which firing finishes first. In the following we show how we solve this problem.

4.2 A Parametric State Space

The parametric state space of G_{ex}^{par} with execution times p and q for actors a and b is given in Figure 3. To simplify the figure, the details of states are not shown. Each dot represents a state. The start and end of firings in each state is denoted by the actor name with subscript s or e respectively. For example a_s shows the start of a firing of actor a .

After starting firings of a and b , v changes to $(\{p\}, \{q\})$. At this stage, a time step equal to the smallest among all elements in the multisets of v must be taken, but the relation between p and q is unknown. Therefore, we split the parameter space into two mutually exclusive parts with $p < q$ and $p > q$. For each of these parts, the state space continues in a separate branch. Since our final goal is finding λ^* and since λ^* is continuous, we do not need to consider the case $p = q$ as the cycle means of this part of the parameter space are covered by expressions obtained by both the cases $p < q$ and $p > q$.

In case $p > q$, the first vertical arrow in the figure, after a time step as large as q , b finishes its firing. So γ and v become $(0, 0, 1, 2)$ and $(\{p - q\}, \{\})$. The execution proceeds by a time step as large as $p - q$ which leads to the end of the firing of a and consequently the start of new firings of actors a and b , changing v to $(\{p\}, \{q\})$. Since in this branch we already assumed that $p > q$, no new partitioning of the parameter space is needed and the execution proceeds by a time step as large as q . The state space in this branch ends in a periodic phase repeating the last two steps. From the periodic phase, we can compute the throughput. The length of the period is $(p - q) + q = p$ and during the period only one firing of actors a and b occurs. Therefore, the throughput is $1/p$ if $p > q$.

We proceed for the case where $p < q$. After a time step as large as p , actor a finishes its firing and starts a new firing. So γ and v become $(0, 1, 0, 0)$ and $(\{p\}, \{q - p\})$ respectively. At this state, the parameter space needs to be split again into two parts: $p < q - p$ and $q - p < p$ (or $2p < q$ and $2p > q$). Note that the state-space exploration only continues if the newly added constraints do not conflict with the previously made assumptions in the earlier states. In this case, both $2p < q$ and $2q < p$ are compatible with $p < q$. The case $2p < q$ gets periodic in a few steps with throughput $1/q$. The other case requires a new partitioning of the parameter space. Each branch continues till either it ends up in its periodic phase or the constraint set contains conflicting constraints. As shown in the figure, the state space of the example continues to repeat a similar pattern. All subsequent branches have the same throughput $1/q$. We can conclude that, as before, $DCMS(G_{ex}^{par}) = \{p, q\}$ for the whole parameter space.

From the example, we can see that the multisets in v contain linear combinations of parameters throughout the execution of the graph. We also observe that the equations

partitioning the parameter space need to be stored in the states.

Definition 7 [Parametric State] The state of a parametric SDFG (A, C, E_p) is a tuple (γ, v, Φ) . γ , as in a regular state a channel state, associates with each channel the amount of tokens present in that channel in that state. Actor status $v : A \rightarrow \mathbb{N}^T$ associates with each actor $a \in A$ a multiset of linear combinations of parameters, each such combination is denoted by a vector $\bar{t}$ in $T = \mathbb{Q}^{d+1}$ containing the coefficients of the linear expression $\bar{t}(\bar{p})$. The state constraint set Φ is a subset of T , which contains all of the assumptions, $\bar{t}(\bar{p}) > 0$, in the form of inequalities on the parameters made so far.

In Figure 3, the horizontal branches of the state space continue to partition the parameter space into ever smaller pieces ad infinitum. The further splitting of the parameter space does not result in the infeasibility of the system of inequalities. This shows that the state space can be infinite.

4.3 Throughput Calculation

Algorithm *coverStateSpace* shows the state-space method for the throughput calculation of a parametric SDFG. It works recursively in a depth-first-search fashion, branching the parameter space as explored. Below we assume that parameter ranges are bounded. The algorithm receives a parametric SDFG G , initial state $s = (\gamma, v, \Phi)$, and a bound on the depth of the recursion D as arguments and returns the DCMS of G . The latter is needed since a parametric state space is potentially infinite. Upon reaching the bound on the depth, the remainder of the parameter space is searched via the divide-and-conquer algorithm explained in the next section. D can thus be used as a control parameter to steer the algorithm towards a (mostly) state-space-based method or a divide-and-conquer method. We use the absence of integer points in the remainder of the parameter space to be explained as an extra criterion to switch to the divide-and-conquer method, as this avoids spending a lot of time in searching increasingly smaller parts of the parameter space.

Algorithm *coverStateSpace*(s, G, D)

Input: A parametric state $s = (\gamma, v, \Phi)$

Input: A strongly connected parametric SDFG $G = (A, C, E_p)$

Input: A bound D on the recursive depth

Output: DCMS of G

```

1.  if ( $D = 0$ )
2.    then Divide&Conquer( $G, \Phi$ );
3.  else for all  $t \in \cup_{a \in A} v(a)$ 
4.    do  $n = (\gamma_n, v_n, \Phi_n) = \text{nextState}(t, s, G)$ ;
5.    if ( $n \neq \text{INT\_INF}$  and not BS)
6.      then if ( $n \in nBList$ )
7.        then  $\bar{e} = \text{calcCMExp}(nBList, n)$ ;
```

```

8.          insert( $DCMS, \bar{e}$ );
9.        else push( $nBList, n$ );
10.       coverStateSpace( $n, G, D - 1$ );
11.     else if ( $n \neq \text{INT\_INF}$ )
12.       then reset( $nBList$ );
13.       coverStateSpace( $n, G, D - 1$ );
14.     else if ( $n \neq \text{INF}$ )
15.       then Divide&Conquer( $G, \Phi_n$ );
```

The algorithm first checks if the maximum execution depth has been reached. If so, it calls Algorithm *Divide&Conquer* of the next section; if not, it uses procedure *nextState* which accepts parameter expression t (an element of a multiset in v) as a clock step, thereby assuming that t is the minimum time that should elapse before any event can occur. Then, *nextState* returns the next state $n = (\gamma_n, v_n, \Phi_n)$ if Φ_n contains integer solutions within the given parameter bounds. For cases where Φ_n lacks integer solutions or it has no solution at all *INT_INF* and *INF* are returned respectively. *nextState* also marks the current state as a branching state (BS) or a non-branching state depending on whether the parameter space is split.

If the algorithm is invoked for state s , for every element $t \in \cup_{a \in A} v(a)$, a new branch is explored, the procedure *nextState* is called and the new state $n = (\gamma_n, v_n, \Phi_n)$ is created. We know that none of the states in a periodic phase can be branching since the constraints in the sets of the recurrent states should be identical. Therefore, the search for a recurrent state only occurs in *nBList*, which stores the non-branching states visited since the last branching state. If Φ_n is feasible and n is non-branching, then the algorithm checks whether the state is recurrent (has already been visited) by comparing it with already stored non-branching states in *nBList*. If the state is recurrent, the algorithm calculates the cycle mean expression, by calling *calcCMExp*. This function finds the length of the periodic phase $|P|$ by adding up the length of clock transitions between the two recurrent states and counts the number of firings $|a|_p$ of an arbitrary actor a in the periodic phase. The throughput can be calculated as $|a|_p / (|P|q(a))$ where q is the repetition vector of the graph. The obtained expression, which is always a dominating mcme, is stored in DCMS. Then the algorithm returns and continues at the last stored branching state if any is left. If n is not recurrent, it is stored in *nBList* and the algorithm is invoked recursively for n , decrementing bound D .

In case n is a branching state and integer solutions are left, the algorithm is invoked recursively after clearing *nBList*. If Φ_n is not infeasible but contains no integer solutions, the *Divide&Conquer* algorithm that is explained in the next section is called. This algorithm finds the mcme of all points in the region Φ_n .

Theorem 8 Given a parametric SDFG G and parameters

PROOF To show that the throughput region associated with $\bar{e}$ is convex, we need to prove that any point $\bar{p} = t\bar{p}_1 + (1-t)\bar{p}_2$ where $0 < t \leq 1$ belongs to the throughput region of $\bar{e}$, i.e., $\bar{e}(\bar{p}) = \lambda^*(\bar{p})$ for every two points $\bar{p}_1$ and $\bar{p}_2$ belonging to the throughput region of $\bar{e}$.

Because $\bar{e}$ is a linear function, if $\bar{e}(\bar{p}_1) = \lambda^*(\bar{p}_1)$ and $\bar{e}(\bar{p}_2) = \lambda^*(\bar{p}_2)$, we have

$$\bar{e}(t\bar{p}_1 + (1-t)\bar{p}_2) = t\bar{e}(\bar{p}_1) + (1-t)\bar{e}(\bar{p}_2). \quad (1)$$

According to the definitions

$$\bar{e}(\bar{p}_1) = \lambda^*(\bar{p}_1) = \max_{\bar{e}_i \in DCMS} \bar{e}_i(\bar{p}_1)$$

and

$$\bar{e}(\bar{p}_2) = \lambda^*(\bar{p}_2) = \max_{\bar{e}_i \in DCMS} \bar{e}_i(\bar{p}_2)$$

for all $\bar{e}_i \in DCMS$, we can write

$$\begin{aligned} \bar{e}_i(\bar{p}_1) &\leq \bar{e}(\bar{p}_1) \\ \bar{e}_i(\bar{p}_2) &\leq \bar{e}(\bar{p}_2). \end{aligned}$$

Therefore, for all $\bar{e}_i \in DCMS$,

$$t\bar{e}_i(\bar{p}_1) + (1-t)\bar{e}_i(\bar{p}_2) \geq t\bar{e}(\bar{p}_1) + (1-t)\bar{e}(\bar{p}_2)$$

which means that

$$\bar{e}(\bar{p}) = \max_{\bar{e}_i \in DCMS} \bar{e}_i(\bar{p}) = \lambda^*(\bar{p}).$$

□

We call these convex polyhedra *throughput regions*. Figure 4 shows two throughput regions for the running example, corresponding to $\bar{e}_1 = q$ and $\bar{e}_2 = p$ within a rectangular area under corner points $\bar{v}_1, \dots, \bar{v}_4$. The following corollary directly follows from Proposition 9.

Corollary 10 *If for every corner point $\bar{v}$ of an arbitrary polyhedron of the parameter space, $\lambda^*(\bar{v}) = \bar{e}(\bar{v})$ for some mcme $\bar{e}$, then for every point $\bar{p}$ in that region, $\lambda^*(\bar{p}) = \bar{e}(\bar{p})$.*

Hence, the parameter space is composed of throughput regions. Suppose $\bar{e}$ is the mcme of an arbitrary interior point of a convex polyhedron C . By comparing the evaluation of $\bar{e}$ for every vertex (corner point) of C and the actual maximum cycle mean value of that vertex, we can detect whether C is a subset of a single throughput region or whether it is covered by parts of more regions. If for any vertex, these two compared values are different, then C is covered by more than one throughput region; otherwise it is part of a single throughput region (namely that of dominating mcme $\bar{e}$). This idea can be used in a divide-and-conquer method if we add a partitioning strategy, to be applied after detecting a region with more than one mcme. Partitioning continues till all the created regions have a single mcme. All obtained mcme together form the DCMS.

Since λ^* is continuous, the mcme of two neighboring regions are valid for all points on the border of the regions. This means that for any two neighboring regions with

mcme $\bar{e}_1$ and $\bar{e}_2$, their border is characterized by the equation $\bar{e}_1(\bar{p}) = \bar{e}_2(\bar{p})$. We address this (hyper) plane as the *splitting plane*; in Figure 4, this is the line $p = q$. In other words, if we have two mcme for two neighboring regions, we can directly calculate the border of the two regions. The following proposition shows that a splitting plane obtained from two different mcme of a convex region always passes through the region and splits the region into smaller ones.

Proposition 11 *Let $\bar{e}_1$ and $\bar{e}_2$ be mcme associated to points $\bar{p}_1$ and $\bar{p}_2$ respectively. If $\bar{e}_1(\bar{p}_1) \neq \bar{e}_2(\bar{p}_1)$ and $\bar{e}_1(\bar{p}_2) \neq \bar{e}_2(\bar{p}_2)$, then $\bar{e}_1(\bar{p}_1) - \bar{e}_2(\bar{p}_1) > 0$ and $\bar{e}_1(\bar{p}_2) - \bar{e}_2(\bar{p}_2) < 0$.*

PROOF Considering the definition of mcme for a point, we have

$$\begin{aligned} \bar{e}_1(\bar{p}_1) &= \max_{\bar{e}_i \in DCMS} \bar{e}_i(\bar{p}_1) \\ \bar{e}_2(\bar{p}_2) &= \max_{\bar{e}_i \in DCMS} \bar{e}_i(\bar{p}_2) \end{aligned}$$

Therefore, since $\bar{e}_1(\bar{p}_1) \neq \bar{e}_2(\bar{p}_1)$ and $\bar{e}_1(\bar{p}_2) \neq \bar{e}_2(\bar{p}_2)$, $\bar{e}_1(\bar{p}_1) > \bar{e}_2(\bar{p}_1)$ and $\bar{e}_2(\bar{p}_2) > \bar{e}_1(\bar{p}_2)$. Therefore,

$$\begin{aligned} \bar{e}_1(\bar{p}_1) - \bar{e}_2(\bar{p}_1) &> 0 \\ \bar{e}_1(\bar{p}_2) - \bar{e}_2(\bar{p}_2) &< 0 \end{aligned}$$

□

Geometrically speaking, Proposition 11 means that the plane characterized by equation $\bar{e}_1(\bar{p}_2) - \bar{e}_2(\bar{p}_2) = 0$ intersects the line connecting $\bar{p}_1$ and $\bar{p}_2$; in other words, $\bar{p}_1$ and $\bar{p}_2$ are on the opposite sides of the plane, and the plane splits the convex region into two smaller and again convex regions. Using Proposition 11 and Corollary 10 we have our complete algorithm if we can find the mcme of a point in the parameter space. This is achieved by adapting the state-space exploration of [7]. The difference with the generic parametric state-space method is that the evaluation of the expressions in the constraint set are known when searching for an mcme for a concrete point and no branching is required.

Some points in the parameter space may have more than one mcme, if different HSDFG cycles happen to be simultaneously critical. In that case, we may get an expression from this method that does not correspond to any real cycle mean expression of the graph because it contains fragments of different cycles. However, our partitioning strategy only works if the expressions relate to real cycle means. So we need to avoid obtaining such expressions. We show that this is a ‘coincidence’, that only happens on the border of throughput regions and can be avoided by selecting a random point from the parameter space.

Proposition 12 *A randomly selected point from the parameter space has only one mcme with probability one.*

PROOF Let $\bar{p}$ be a randomly selected point in the parameter space. If two different $\bar{e}_1$ and $\bar{e}_2$ are both valid mcme for $\bar{p}$, then the

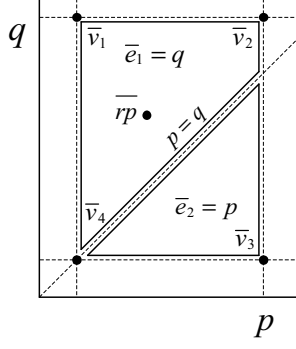


Figure 4. Divide-and-Conquer Method.

evaluation of both $\bar{e}_1$ and $\bar{e}_2$ for $\bar{p}$ must be equal. Therefore, we have

$$\bar{e}_1(\bar{p}) = \bar{e}_2(\bar{p}).$$

Suppose we denote the j th coefficient of expression e_i , $i = 1, 2$, with $\bar{e}_{i,j}$ and of $\bar{p}$ with $\bar{p}_j$. Now, knowing that the constants in $\bar{e}_1$ and $\bar{e}_2$ must be equal, we can rewrite the above equation in the following form.

$$\sum_{j=1}^d (\bar{e}_{1,j} - \bar{e}_{2,j}) \bar{p}_j = 0$$

We know that $\bar{p}$ is randomly selected from a uniformly continuous distribution. Therefore, $\sum_{j=1}^d (\bar{e}_{1,j} - \bar{e}_{2,j}) \bar{p}_j$ is also a random variable in a continuous space. The probability of a continuous random variable being a constant number is zero. Therefore, with probability one, $\bar{e}_1(\bar{p}) \neq \bar{e}_2(\bar{p})$ and $\bar{e}_1$ and $\bar{e}_2$ are not both valid for $\bar{p}$. $\square$

Every convex region can be represented in two different ways using half spaces (H-representation) or vertices of the convex region (V-representation) and these two representations are convertible in a very efficient way [4]. In our algorithm, we use both representations, the V-representation for finding the vertices, and the H-representation for calculating the splitting plane. As before, we assume that parameter ranges are bounded. Algorithm *Divide&Conquer*, given below, receives G and a convex region CR as input. Initially, when applying the algorithm to the entire parameter space, CR is a d -dimensional box obtained by the ranges of the parameters. In Line 4 and 5, all the cycle mean values of all the vertices of CR are checked for the validity of the mcme obtained for a random point $\bar{r}\bar{p}$ in the interior of CR . $Th(\bar{v}_i)$ is the throughput of G for point $\bar{v}_i$, obtained using the non-parametric state-space method of [7]. In case the mcme of $\bar{r}\bar{p}$ is not valid for a vertex $\bar{v}_i$, then the splitting plane obtained from mcme of $\bar{v}_i$ and $\bar{r}\bar{p}$ splits CR (illustrated in Figure 4 for the example with $\bar{v}_3$ in the role of $\bar{v}_i$) by adding half-spaces characterized by vectors $\bar{e}_{r\bar{p}} - \bar{e}_{v_i}$ and $\bar{e}_{v_i} - \bar{e}_{r\bar{p}}$ to CR , respectively. Then the algorithm is invoked for both subregions in Lines 9 and 10. Procedure *ranCornerExpr* receives a vertex $\bar{v}_i$, expression $\bar{e}_{r\bar{p}}$, and $\bar{r}\bar{p}$.

It produces the mcme valid in $\bar{v}_i$ which will be different from $\bar{e}_{r\bar{p}}$. Note that inside this procedure, instead of using $\bar{v}_i$ itself which is typically on the border between throughput regions, for the reason explained, a randomly selected point in its neighborhood on the line through $\bar{v}_i$ and $\bar{r}\bar{p}$ is used instead. In fact, in our algorithm this point is chosen as the first point on this line with different associated mcme from that of $\bar{r}\bar{p}$ by moving in a zeno-fashion toward $\bar{v}_i$. Of course, this is our choice in this algorithm and any other arbitrary point in the region with different mcme is suitable for constructing the splitting plane as well. The algorithm is guaranteed to terminate, because there is a finite number of mcmes and hence a finite number of borders between regions exist that can be used for splitting. The correctness of the algorithm is proven in the following theorem.

Theorem 13 *Given a parametric SDFG G and a parameter space CR , algorithm *Divide&Conquer* finds all the mcmes of parameter space CR and it terminates.*

PROOF We prove the correctness of this function by induction on the number of throughput regions of CR . The base case for the induction is when all the corner points of CR have the same mcme as a randomly generated point $\bar{e}_{r\bar{p}}$ in the interior of CR . Then, due to the convexity of the throughput region, proven by Proposition 9 we know that only one mcme is valid for CR , i.e., CR is part of only one throughput region. In this case, Algorithm *Divide&Conquer* calculates this mcme and terminates immediately. This check is done in the for-loop of Line 4. In case that at least one of the corner points has a different mcme, then the splitting plane made by the mcme of $\bar{e}_{r\bar{p}}$ and the mcme of another point in the interior of the mcme splits CR . This splitting plane due to Proposition 11 cuts CR into two smaller regions, CR_1 and CR_2 , in Lines 7 and 8. Both CR_1 and CR_2 must have at least one throughput region less than CR , because we know that the splitting plane used for cutting CR into CR_1 and CR_2 , is constructed by removing an mcme from both CR_1 and CR_2 . This implies that the used mcme from CR_1 cannot be dominating in CR_2 and vice versa. In other words, CR_1 lacks the throughput region related to the mcme of CR_1 which is used for the splitting plane. Similar reasoning is valid for CR_2 as well. By induction, we know that

the mcmees of both regions CR_1 and CR_2 are found in the recursive calls. The final set of mcmees is the union of the expressions found in each of the recursive calls.

We know that the HSDFG corresponding to G has only a finite number of simple cycles and that each mcme relates to a cycle in this HSDFG. Therefore, there are only a finite number of different mcmees. Therefore, the algorithm terminates. $\square$

Algorithm *Divide&Conquer*(G, CR)

Input: A strongly connected parametric SDFG G

Input: A convex region CR

Output: DCMS of G

1. Let $\overline{rp}$ be a random point in CR ;
2. $\overline{e}_{rp} \leftarrow \text{findMCME}(G, \overline{rp})$;
3. $\text{insert}(\text{DCMS}, \overline{e}_{rp})$;
4. **for** all vertices $\overline{v}_i \in CR$
5. **if** $(\overline{e}_{rp}(\overline{v}_i) \neq 1/Th(v_i))$
6. **then** $\overline{e}_{v_i} = \text{ranCornerExpr}(\overline{v}_i, \overline{e}_{rp}, \overline{rp})$;
7. $CR_1 \leftarrow CR \cup \{\overline{e}_{rp} - \overline{e}_{v_i}\}$;
8. $CR_2 \leftarrow CR \cup \{\overline{e}_{v_i} - \overline{e}_{rp}\}$;
9. $\text{Divide\&Conquer}(G, CR_1)$;
10. $\text{Divide\&Conquer}(G, CR_2)$;

6 Experiments

We have evaluated the execution times of our algorithms using SDFG models of seven real applications. We used the benchmark of [7], consisting of an H.263 decoder, an MP3 decoder, a modem, a satellite receiver, and a sample-rate converter. We further added an H.263 encoder [11] and an MP3 playback application [18]. In *Divide&Conquer*, the CDDLib library [4] is used for all polyhedra operations. In *coverStateSpace*, all operations related to linear inequality systems have been done using LPSolve [1]. All experiments were performed on a P4 PC running at 3.4 Ghz.

In each graph, to each actor with varying execution times a parameter has been assigned. Actors with constant execution times received fixed execution times. In cases where more than one copy of an actor existed in the SDFG, the same parameter was dedicated to all copies. Two experiments with the same parameter set and different ranges for the parameters have been carried out. We used two different ranges for parameters with the same lower-bounds and upper-bounds as large as 110% and 150% of the lower-bounds. These ranges were chosen in line with the worst-case estimates of the execution times of the benchmarks, if any were given. The results of these experiments are shown in Table 1 in two different columns. For each experiment, for each graph, the time for both the state-space method (st) and divide-and-conquer (dc) in seconds, as well as the number of expressions in the DCMS (#e) are shown. In all cases, only very few dominating mcmees (up to 3) have been found, which is a good indication for the simplicity of the resulting throughput expression.

Table 1. Experimental results

	#pa	#act	rep	110%			150%		
				st[s]	dc[s]	#e	st[s]	dc[s]	#e
H.263 decoder	4	4	1190	0.854	0.590	1	0.862	0.589	1
H.263 encoder	5	5	201	0.119	0.212	1	0.241	0.211	1
modem	7	16	48	51	0.568	1	168	0.570	1
MP3 decoder	8	14	14	0.196	0.889	1	0.253	0.896	1
MP3 playback	1	4	10601	8.643	1.268	1	17	8.177	2
sample-rate conv.	4	6	612	102	1.040	2	266	1.246	2
satellite rec.	9	23	4515	-	480	3	-	450	3

The number of parametric execution times (#pa), the number of actors (#act) and the sum of their repetition vector entries (rep, which is also the number of actors in the equivalent HSDFG) of each graph is shown. Since the number of cycles in the equivalent HSDFG directly corresponds to the number of different cycle mean expressions, the sum of repetition vector entries is an important indication for the expected run-time, besides the actor and parameter counts.

We only compared the *Divide&Conquer* and *coverStateSpace* algorithms. The reason is that the *HSDFG method* works on the HSDFGs, and even though we have implemented the fastest cycle enumeration algorithm [8], the algorithm takes generally too long. It only worked for the MP3 decoder for which it only took few milliseconds to compute the DCMS.

The two methods compared in Table 1 are fast in most cases. The divide-and-conquer method is fast in all the cases. It is also less sensitive to the ranges of parameters than the state-space method. However, its execution time does scale up exponentially with an increasing number of parameters. However, typically, in practical applications only a few parameters are needed since the number of actors with varying execution times is limited and the variations can be captured by the same underlying parameters.

The state-space method works very fast for applications like the H.263 decoder, the H.263 encoder, the MP3 playback and the MP3 decoder, which have a few actors with large execution times. In a few cases, it is faster than divide and conquer. On the other hand, it performs poorly on graphs whose actors have approximately equal execution times. For example, for the satellite receiver, the algorithm took more than a few hours.

Summarizing, the results show that a design-time parametric throughput analysis is feasible. Considering the results in, for example, the context of a run-time resource- or quality management application as proposed e.g. in [14], it is clear that the processing time and memory usage of a throughput calculation for concrete values of the execution time parameters, consisting of an evaluation of the maximum value of the obtained dominating mcmees, are negligible compared to the processing time and memory usage of the typical streaming application. They are in general also small compared to the processing time and memory usage

of a traditional throughput calculation, which is typically too expensive to perform at run-time.

7 Conclusion

We have extended throughput analysis of SDFGs to parametric SDFGs so that actors can have parameters as their execution times. The throughput of such graphs is a function of the parameters. Evaluating these functions is much faster than traditional throughput analysis methods. We adapted existing methods for computing throughput to parametric SDFGs and proposed a new, faster, algorithm.

References

- [1] M. Berkelaar, K. Eikland, and P. Notebaert. Ipsolve. C library, <http://www.geocities.com/lpsolve/>.
- [2] A. Dasdan. Experimental analysis of the fastest optimum cycle ratio and mean algorithms. *ACM Transactions on Design Automation of Electronic Systems*, 9(4):385–418, October 2004.
- [3] H. Edelsbrunner, L. Guibas, and M. Sharir. The upper envelope of piecewise linear functions: Algorithms and applications. *Discrete & Computational Geometry*, 4:311–336, 1989.
- [4] K. Fukuda. cddlib. C++ library, Swiss Federal Institute of Technology, http://www.ifor.math.ethz.ch/~fukuda/cdd_home/cdd.html.
- [5] S. Garg and D. Marculescu. System-level process variation driven throughput analysis for single and multiple voltage-frequency island designs. In *Conference on Design Automation and Test in Europe, DATE 07, Proceedings*, pages 403–408. IEEE, 2007.
- [6] A. Ghamarian, M. Geilen, T. Basten, B. Theelen, M. Mousavi, and S. Stuijk. Liveness and boundedness of synchronous data flow graphs. In *6th International Conference on Formal Methods in Computer Aided Design, FMCAD 06, Proceedings*, pages 68–75. IEEE, 2006.
- [7] A. Ghamarian, M. Geilen, S. Stuijk, T. Basten, A. Moonen, M. Bekooij, B. Theelen, and M. Mousavi. Throughput analysis of synchronous data flow graphs. In *6th International Conference on Application of Concurrency to System Design, ACSD 06, Proceedings*, pages 25–36. IEEE, 2006.
- [8] D. Johnson. Finding all the elementary circuits of a directed graph. *SIAM Journal on Computing*, 4(1):77–84, 1975.
- [9] R. Karp and R. Miller. Properties of a model for parallel computations: Determinacy, termination, queueing. *SIAM Journal on Applied Mathematics*, 14(6):1390–1411, November 1966.
- [10] E. Lee. *A Coupled Hardware and Software Architecture for Programmable Digital Signal Processors*. PhD thesis, University of California, Berkeley, June 1986.
- [11] H. Oh and S. Ha. Fractional rate dataflow model for efficient code synthesis. *Journal of VLSI Signal Processing*, 37(1):41–51, 2004.
- [12] J. Pino, S. Bhattacharyya, and E. Lee. A hierarchical multiprocessor scheduling system for DSP applications. In *Conference on Signals, Systems and Computers, Proceedings*, pages 122–126. IEEE, 1995.
- [13] P. Poplavko, T. Basten, M. Bekooij, J. van Meerbergen, and B. Mesman. Task-level timing models for guaranteed performance in multiprocessor networks-on-chip. In *International Conference on Compilers, Architecture, and Synthesis for Embedded Systems, CASES 03, Proceedings*, pages 63–72. ACM, 2003.
- [14] P. Poplavko, T. Basten, and J. van Meerbergen. Execution-time prediction for dynamic streaming applications with task-level parallelism. In *10th Euromicro Conference on Digital System Design, DSD 07, Proceedings*, pages 228–235. IEEE, 2007.
- [15] S. Sriram and S. Bhattacharyya. *Embedded Multiprocessors: Scheduling and Synchronization*. Marcel Dekker, Inc, New York, NY, USA, 2000.
- [16] S. Stuijk. *Predictable Mapping of Streaming Applications on Multiprocessors*. Ph.D. dissertation, Eindhoven University of Technology, 2007.
- [17] S. Stuijk, T. Basten, M. Geilen, and H. Corporaal. Multiprocessor resource allocation for throughput-constrained synchronous dataflow graphs. In *44th Design Automation Conference, DAC 07, Proceedings*, pages 777–782. ACM, 2007.
- [18] M. Wiggers, M. Bekooij, and G. Smit. Efficient computation of buffer capacities for cyclo-static dataflow graphs. In *44th Design Automation Conference, DAC 07, Proceedings*, pages 658–663, New York, NY, USA, 2007. ACM.